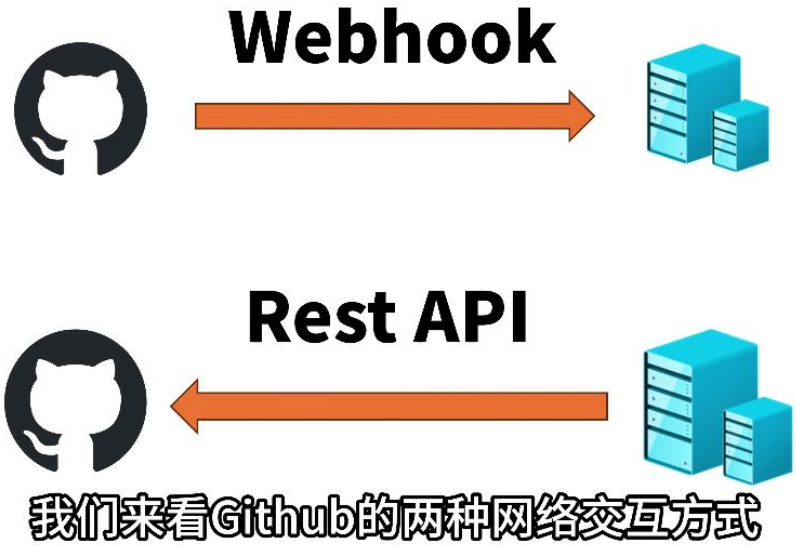
-- 以下为AI生成的图文笔记内容 --

一、网页钩子与Rest API 00:00

1. 网页钩子 00:06



-
- **定义**：Webhook直译为"网络钩子"，是一种事件通知机制
- **工作原理**：当特定事件发生时，GitHub会通过HTTP接口调用方式通知到指定服务器
- **特点**：GitHub作为客户端调用我们的服务器，通知事件如代码提交、PR创建等

1）网络钩子作为客户端 00:33

-
- **服务器要求**：

- 需要公网IP的云服务器
- 推荐使用Ubuntu系统
- 需要安装Python3和虚拟环境工具

- **环境配置**：
  - 创建虚拟环境：python3 -m venv web_hook_venv
  - 激活环境：source web_hook_venv/bin/activate
  - 安装依赖：pip3 install fastapi uvicorn python-multipart

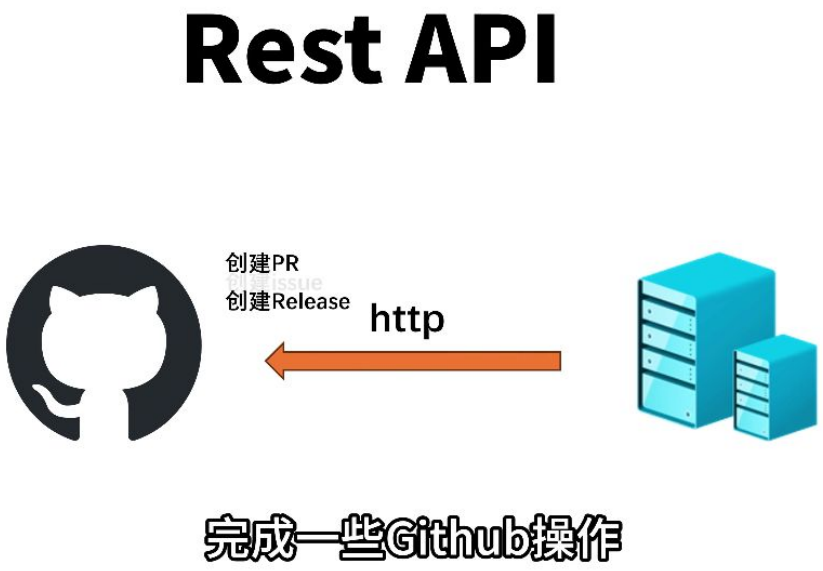
-
- **代码实现**：

```
from fastapi import FastAPI, Request

import uvicorn

app = FastAPI()

@app.post("/webhook")

async def print_json(request: Request):

    print(await request.form())

    return {"message": "Data received and printed"}
```

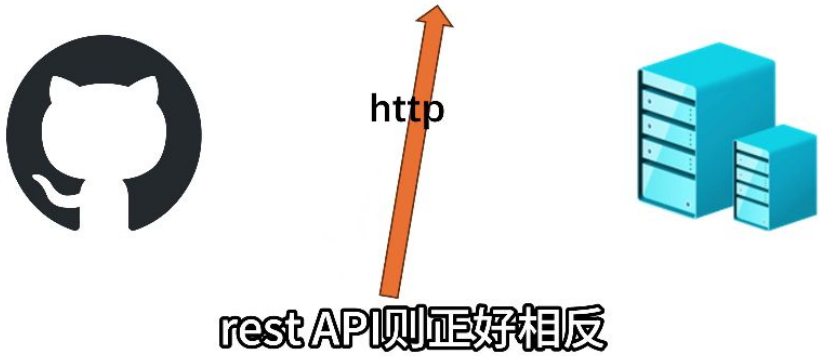
```
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=80)
```

- **功能**：监听80端口的/webhook路径，打印接收到的表单数据

2）网络钩子作为服务端 03:35

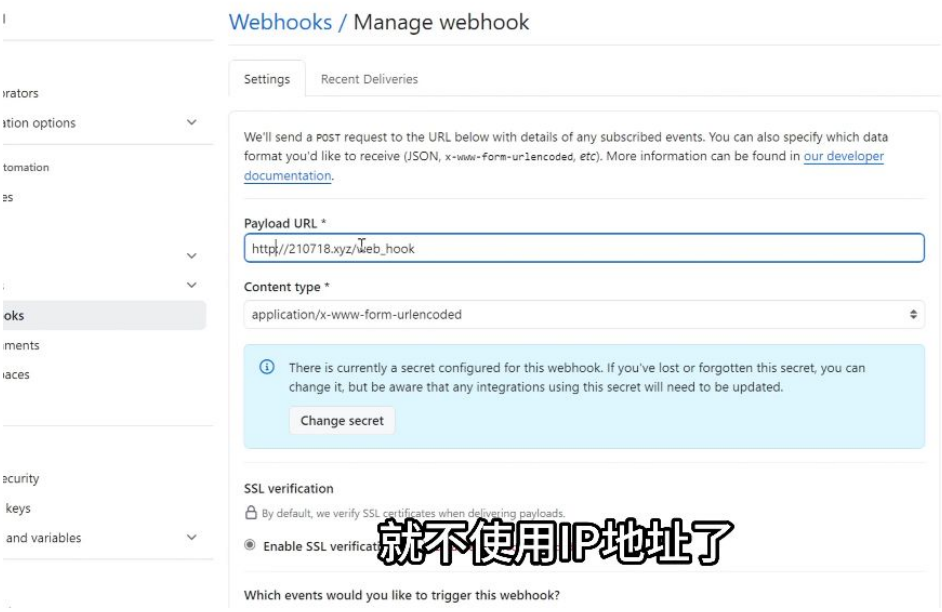


-
- **配置步骤**：
    - 进入仓库Settings → Webhooks → Add webhook
    - 填写Payload URL（如http://服务器IP/web_hook）
    - 选择Content type为application/x-www-form-urlencoded
    - 设置Secret密钥用于验证请求来源
    - 选择触发事件（如仅push事件）



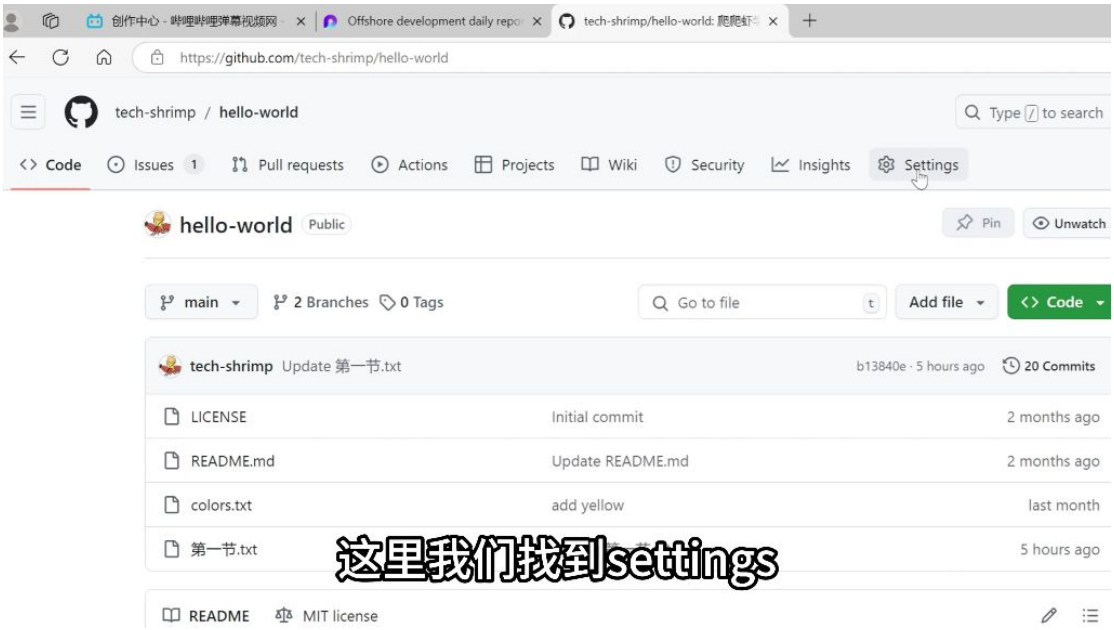
-
- **安全验证**：
  - GitHub会在请求头中包含X-Hub-Signature-256签名
  - 服务端需使用相同密钥对payload进行sha256哈希计算验证
  - 验证代码示例：

```
def verify_signature(request_body, secret_token, signature_header):
    hash_object = hmac.new(secret_token.encode('utf-8'),
                           msg=request_body,
                           digestmod=hashlib.sha256)
    expected_signature = "sha256=" + hash_object.hexdigest()
    if not hmac.compare_digest(expected_signature, signature_header):
        raise Exception("Request signatures didn't match!")
```

- 

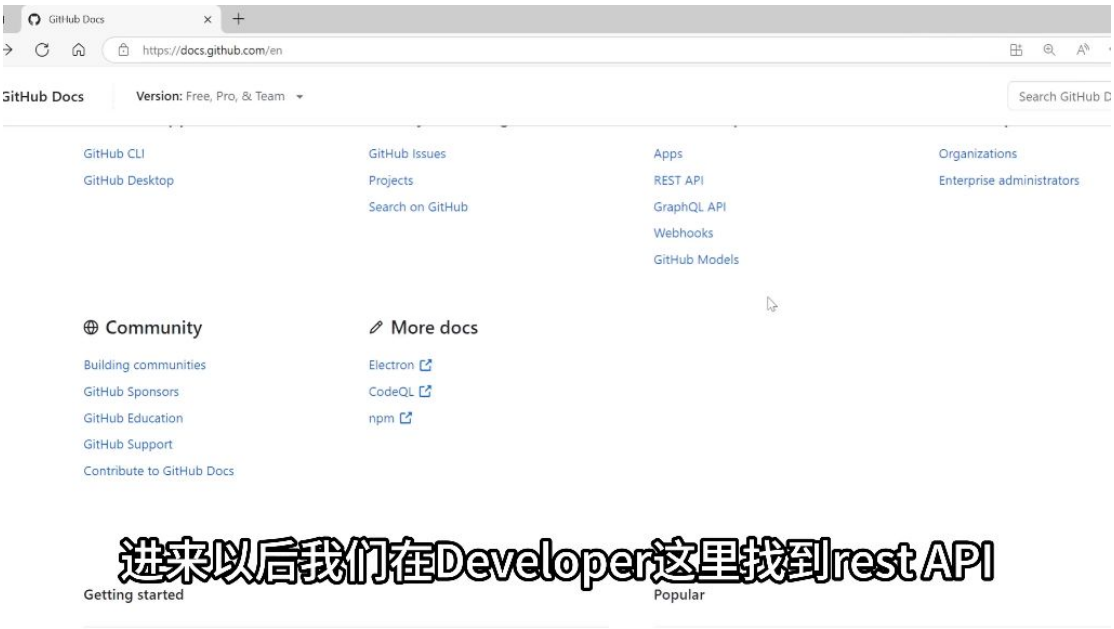
- **进阶配置**：
  - 使用域名替代IP地址
  - 通过Cloudflare等DNS服务商配置HTTPS
  - 免费获取SSL证书提高安全性

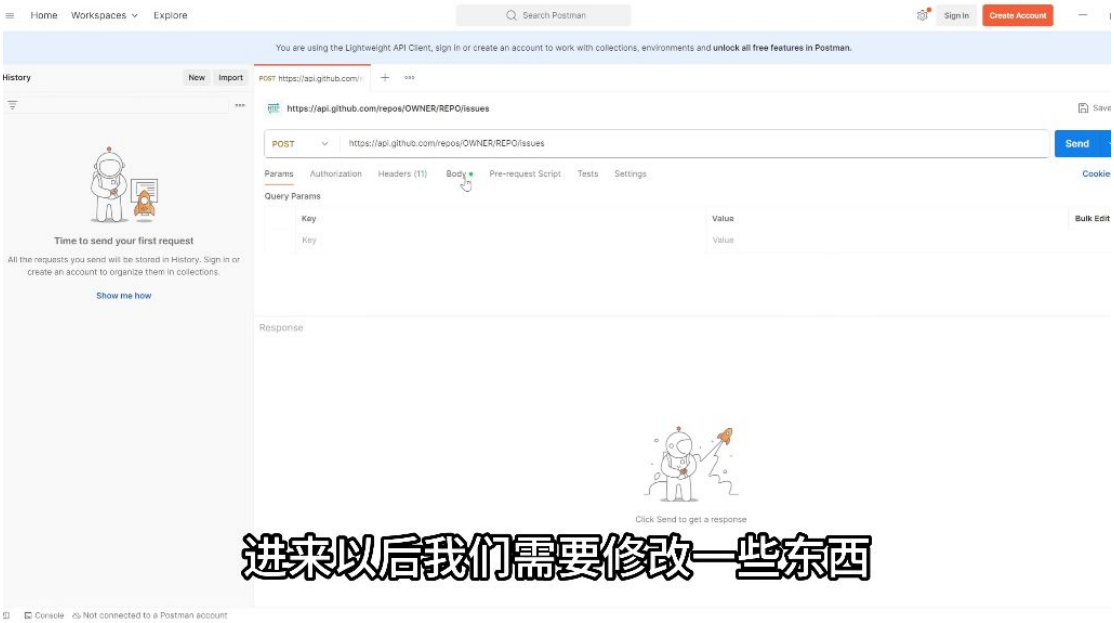
2. Rest API 10:24

- 

- **定义**：GitHub提供的一组API接口
- **特点**：与Webhook相反，我们作为客户端调用GitHub的API
- **用途**：实现自动化操作替代网页手动操作

1）创建问题 10:55

- 

- **访问方式**：通过docs.github.com/en/rest获取API文档
- **认证方式**：使用Personal Access Token
  - 生成路径：Settings → Developer settings → Personal access tokens
  - 权限设置：根据需要选择仓库读写权限
- 

- **调用示例**：

curl -L \

  -X POST \

-H "Accept: application/vnd.github+json" \

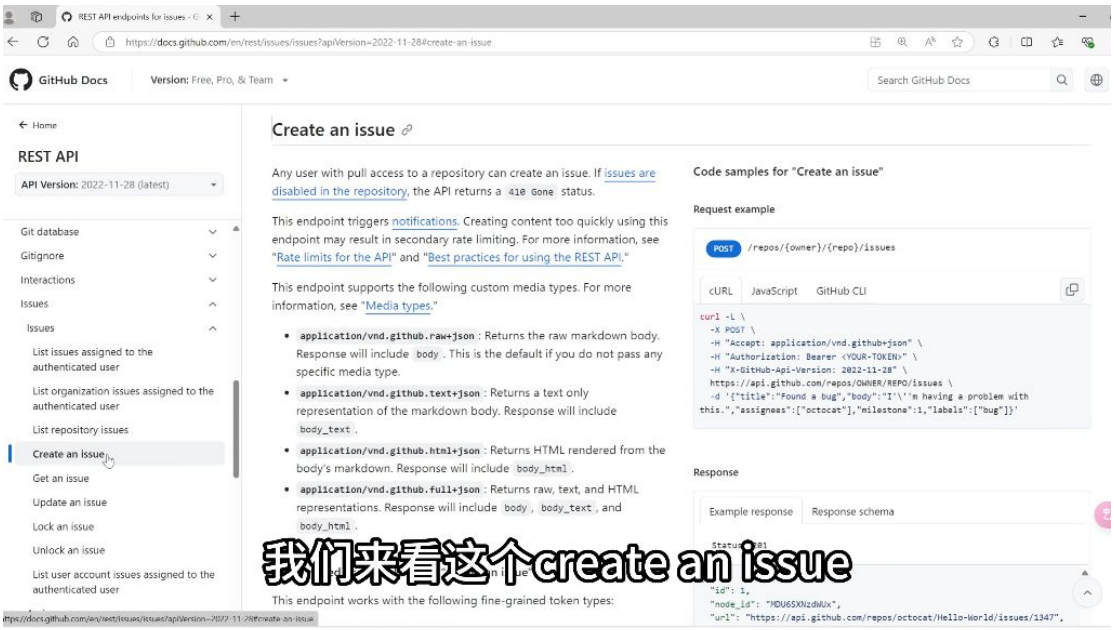
-H "Authorization: Bearer <YOUR-TOKEN>" \

-H "X-GitHub-Api-Version: 2022-11-28" \

https://api.github.com/repos/OWNER/REPO/issues \

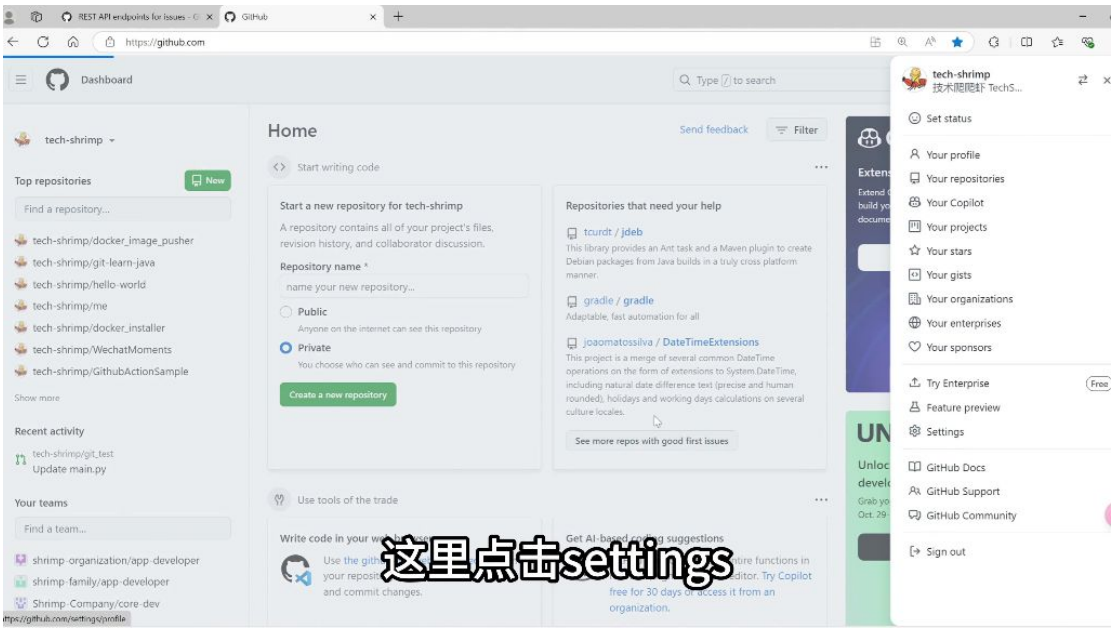
-d '{"title":"Found a bug","body":"I'\''m having a problem with this.","assignees":["octocat"],"labels":["bug"]}'

2）发布 11:15



-
- **API端点**：POST /repos/{owner}/{repo}/releases
- **必要参数**：
  - tag_name：版本标签（如v1.0.0）
  - target_commitish：目标分支（如master）
  - name：发布名称
  - body：发布说明

3）问题 12:38

- 

- **注意事项**：
  - Token生成后需立即复制，页面刷新后将无法查看
  - 设置合理的过期时间（如7天）
  - 根据操作需求精确分配权限

3. 使用代码调用API 14:42

- **常用库**：
  - Python：requests库
  - JavaScript：axios或fetch API
  - 命令行：curl工具
- **最佳实践**：
  - 妥善保管Access Token
  - 处理API速率限制
  - 使用try-catch处理异常
  - 记录API调用日志

二、知识小结

知识点	核心内容	技术要点/易混淆点	应用场景
GitHub Webhook	事件驱动的通知机制，GitHub作为客户端调用用户服务器	**与REST API方向相反** • 需部署HTTP服务端 • 支持SHA256签名验证 • 支持80/443端口	• 代码提交通知 • PR创建提醒 • 自动触发CI/CD
REST API	GitHub提供的编程接口，用户作为客户端调用	**认证方式**：Personal Access Token • 需配置权限范围 • 支持CRUD操作	• 自动创建issue • 发布release版本 • 仓库管理自动化
Webhook配置	需在GitHub仓库Settings中设置	**关键参数**： • 回调URL • Content-Type • **Secret密钥** • 事件类型筛选	企业级应用需结合： • HTTPS加密 • 域名解析(如Cloudflare)
API调用示例	通过Postman演示API操作	**典型流程**： 1. 生成Token 2. 设置请求头 3. 构造JSON body 4. 处理响应	• issue管理自动化 • 版本发布流水线 • 仓库数据分析
安全机制	Webhook签名验证流程	**验证原理**： 1. GitHub用secret对payload签名 2. 服务端重复哈希计算 3. 比对X-Hub-Signature	防止伪造请求的**核心保障**
开发工具链	Python+FastAPI实现方案	**技术栈对比**： • 虚拟环境(pipenv) • 异步框架(FastAPI)	快速构建轻量级Webhook接收服务

		• 多语言支持 (Java/Node.js)	